

NotebookLM Source: Template Completo de Código (Arquitetura)

Este documento serve como referência de código-fonte funcional para todas as camadas que devem ser implementadas na avaliação.

1. domain.py (Modelo de Domínio Puro)

Sem acoplamento com banco de dados, contendo classes de negócio puras, entidades, objetos de valor e agregação.

```
from dataclasses import dataclass
from datetime import date
from typing import List, Optional

# Exemplo de Objeto de Valor (Value Object) - Imutável por igualdade de valor
@dataclass(frozen=True)
class OrderLine:
    orderid: str
    sku: str
    qty: int

# Exemplo de Entidade (Entity) - Identificada pelo atributo único "reference"
class Batch:
    def __init__(self, ref: str, sku: str, qty: int, eta: Optional[date] = None):
        self.reference = ref
        self.sku = sku
        self._initial_qty = qty
        self.eta = eta
        self._allocations: set[OrderLine] = set()
        self.events: List = [] # Rastreamento de eventos de domínio

    def allocate(self, line: OrderLine):
        if self.can_allocate(line):
            self._allocations.add(line)
            # Evento de domínio opcional
            # self.events.append(events.Allocated(orderid=line.orderid, sku=line.sku, qty=line.qty,
            ...))

    def deallocate(self, line: OrderLine):
        if line in self._allocations:
            self._allocations.remove(line)

    @property
    def allocated_quantity(self) -> int:
        return sum(line.qty for line in self._allocations)

    @property
    def available_quantity(self) -> int:
        return self._initial_qty - self.allocated_quantity

    def can_allocate(self, line: OrderLine) -> bool:
        return self.sku == line.sku and self.available_quantity >= line.qty

    def __eq__(self, other):
        if not isinstance(other, Batch):
            return False
        return self.reference == other.reference

    def __hash__(self):
        return hash(self.reference)
```

2. messages.py (Comandos e Eventos)

Contém a definição dos sinais de tráfego de dados da aplicação.

```
from dataclasses import dataclass
from datetime import date
from typing import Optional

# Classe base para mensagens
class Message:
    pass

# Classe base para comandos (Ação imperativa que pode falhar)
@dataclass
class Command(Message):
    pass

# Classe base para eventos (Fato histórico que já aconteceu)
@dataclass
class Event(Message):
    pass

# --- COMANDOS ---
@dataclass
class CreateBatch(Command):
    ref: str
    sku: str
    qty: int
    eta: Optional[date] = None

@dataclass
class Allocate(Command):
    orderid: str
    sku: str
    qty: int

# --- EVENTOS ---
@dataclass
class Allocated(Event):
    orderid: str
    sku: str
    qty: int
    batchref: str

@dataclass
class OutOfStock(Event):
    sku: str
```

3. repository.py (Camada de Repositório)

Abstração sobre a persistência dos dados.

```
import abc
from domain import Batch

class AbstractRepository(abc.ABC):
    @abc.abstractmethod
    def add(self, batch: Batch) -> None:
        raise NotImplementedError
```

```

    @abc.abstractmethod
    def get(self, reference: str) -> Batch:
        raise NotImplementedError

    @abc.abstractmethod
    def list(self) -> list[Batch]:
        raise NotImplementedError

class SQLAlchemyRepository(AbstractRepository):
    def __init__(self, session):
        self.session = session

    def add(self, batch: Batch) -> None:
        self.session.add(batch)

    def get(self, reference: str) -> Batch:
        return self.session.query(Batch).filter_by(reference=reference).first()

    def list(self) -> list[Batch]:
        return self.session.query(Batch).all()

```

4. database.py (Mapeamento Imperativo)

Configuração do banco e acoplamento desacoplado das tabelas com as classes do domínio.

```

from sqlalchemy import Table, Column, Integer, String, Date, ForeignKey, MetaData
from sqlalchemy.orm import registry, relationship
import domain

metadata = MetaData()
mapper_registry = registry()

# Definição física das tabelas
batches_table = Table(
    "batches",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("reference", String(255), unique=True, nullable=False),
    Column("sku", String(255), nullable=False),
    Column("qty", Integer, nullable=False),
    Column("eta", Date, nullable=True),
)

order_lines_table = Table(
    "order_lines",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("sku", String(255), nullable=False),
    Column("qty", Integer, nullable=False),
    Column("orderid", String(255), nullable=False),
)

allocations_table = Table(
    "allocations",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("batch_id", ForeignKey("batches.id")),
    Column("orderline_id", ForeignKey("order_lines.id")),
)

def start_mappers():
    # Mapeando OrderLine (Value Object/Entidade auxiliar)
    lines_mapper = mapper_registry.map_imperatively(

```

```

        domain.OrderLine,
        order_lines_table
    )

    # Mapeando Batch (Entidade Raiz) ligando as alocações via tabela associativa
    mapper_registry.map_imperatively(
        domain.Batch,
        batches_table,
        properties={
            "_initial_qty": batches_table.c.qty, # Mapeia atributo privado para coluna qty
            "_allocations": relationship(
                lines_mapper,
                secondary=allocations_table,
                collection_class=set, # Armazena em um set do Python
            )
        }
    )
)

```

5. handlers.py (Casos de Uso / Camada de Serviço)

Lida com o recebimento de Comandos e publica Eventos através do UOW.

```

import domain
import messages
from unit_of_work import AbstractUnitOfWork

def add_batch(cmd: messages.CreateBatch, uow: AbstractUnitOfWork):
    with uow:
        # 1. Cria a entidade usando regras de domínio
        batch = domain.Batch(ref=cmd.ref, sku=cmd.sku, qty=cmd.qty, eta=cmd.eta)
        # 2. Adiciona ao repositório exposto pelo UOW
        uow.batches.add(batch)
        # 3. Faz o commit da transação
        uow.commit()

def allocate(cmd: messages.Allocate, uow: AbstractUnitOfWork) -> str:
    line = domain.OrderLine(orderid=cmd.orderid, sku=cmd.sku, qty=cmd.qty)
    with uow:
        batches = uow.batches.list()
        # Validação do domínio
        available_batches = [b for b in batches if b.can_allocate(line)]
        if not available_batches:
            # Lança exceção de domínio se indisponível
            raise ValueError(f"Fora de estoque para o SKU: {line.sku}")

        # Seleciona o melhor lote (ex: com menor data de chegada - ETA)
        sorted_batches = sorted(available_batches, key=lambda b: b.eta or date.min)
        selected_batch = sorted_batches[0]

        # Executa lógica de domínio
        selected_batch.allocate(line)
        uow.commit()
        return selected_batch.reference

```

6. read_model.py (Modelo de Leitura / CQRS)

Consultas diretas e eficientes no banco sem regras de domínio.

```
# Consulta otimizada e direta para o GET /view-allocations
def get_allocations(orderid: str, session) -> list[dict]:
    # Faz join direto nas tabelas físicas usando SQL cru ou SQLAlchemy básico
    query = """
        SELECT b.reference as batchref, ol.sku, ol.qty
        FROM allocations a
        JOIN batches b ON a.batch_id = b.id
        JOIN order_lines ol ON a.orderline_id = ol.id
        WHERE ol.orderid = :orderid
    """
    result = session.execute(query, {"orderid": orderid})
    return [dict(row) for row in result]
```